Sorting Algorithms and Practice Questions

Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order.

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

```java
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Selection Sort

Selection Sort finds the minimum element from the unsorted portion and places it at the beginning.

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

```java
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Swap the minimum element with the first element
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
```

Insertion Sort

Insertion Sort builds the final sorted array one item at a time by inserting each element into its correct position.

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

```java
public static void insertionSort(int[] arr) {
    int n = arr.length;
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Merge Sort

Merge Sort is a divide-and-conquer algorithm that divides the array into halves, sorts them, and then merges them back together.

Time Complexity: O(n log n)
Space Complexity: O(n)

java

```java
public static void mergeSort(int[] arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

public static void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    for (int i = 0; i < n1; i++)
        leftArray[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArray[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        } else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = leftArray[i];
        i++;
        k++;
    }

    while (j < n2) {
```

```
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}
```

Quick Sort

Quick Sort is a divide-and-conquer algorithm that picks a pivot element and partitions the array around the pivot.

Time Complexity: O(n log n) average, O(n^2) worst case
Space Complexity: O(log n)

```java
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}
```

Practice Questions on Sorting

Question 1: Array Sorting Analysis

Given the array: [64, 34, 25, 12, 22, 11, 90]

Trace through the first two iterations of Bubble Sort and show the state of the array after each iteration.

Question 2: Selection Sort Implementation

Implement Selection Sort to sort an array of strings in alphabetical order.

Question 3: Insertion Sort Optimization

Modify the Insertion Sort algorithm to sort in descending order instead of ascending order.

Question 4: Merge Sort Understanding

Explain why Merge Sort has O(n log n) time complexity. Draw the recursion tree for an array of size 8.

Question 5: Quick Sort Pivot Selection

Compare the performance of Quick Sort when the pivot is always the first element versus when the pivot is the middle element. Use the array [1, 2, 3, 4, 5, 6, 7, 8].

Question 6: Sorting Algorithm Comparison

Create a table comparing Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort based on:

- Best case time complexity
- Worst case time complexity
- Space complexity
- Stability

Multiple Choice Quiz

1. What is the time complexity of Bubble Sort in the worst case? a) O(n) b) O(n log n) c) $O(n^2)$ d) $O(2^n)$

Answer: c) $O(n^2)$

2. Which sorting algorithm is stable and has O(n log n) time complexity? a) Quick Sort b) Selection Sort c) Merge Sort d) Bubble Sort

Answer: c) Merge Sort

3. In Quick Sort, what is the purpose of the partition function? a) To divide the array into two equal halves b) To find the minimum element c) To rearrange elements around a pivot d) To merge two sorted arrays

Answer: c) To rearrange elements around a pivot

4. Which sorting algorithm performs best on nearly sorted data? a) Selection Sort b) Insertion Sort c) Bubble Sort d) Quick Sort

Answer: b) Insertion Sort

5. What is the space complexity of Merge Sort? a) O(1) b) O(log n) c) O(n) d) O(n^2)

Answer: c) O(n)

Practice Coding Questions

1. Sort Colors Problem Given an array with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue. Use integers 0, 1, and 2 to represent red, white, and blue respectively.

```java
public void sortColors(int[] nums) {
    // Your implementation here
}
```

2. Kth Largest Element Find the kth largest element in an unsorted array using a sorting-based approach.

```java
public int findKthLargest(int[] nums, int k) {
    // Your implementation here
}
```

3. Sort Array by Frequency Given an array of integers, sort the array according to frequency of elements. Elements with higher frequency should come first.

```java
public int[] frequencySort(int[] nums) {
    // Your implementation here
}
```

4. Merge Intervals Given an array of intervals, merge all overlapping intervals and return an array of non-overlapping intervals.

```java
public int[][] merge(int[][] intervals) {
    // Your implementation here
}
```

5. Custom Comparator Sort Sort an array of strings based on their length. If two strings have the same length, sort them alphabetically.

```java
public String[] sortByLength(String[] strings) {
    // Your implementation here
}
```